

Detailed Requirements — Enterprise E-Commerce & Order Management Platform

A production-grade Spring Boot microservices learning project covering REST APIs, service-to-service communication, Kafka, Saga, Outbox, idempotency, resilience, security, observability, Docker, Kubernetes and CI/CD.

1. High-Level System

The platform consists of an API Gateway and independently deployable business services: User/Identity, Product, Cart, Order, Inventory, Payment, Shipping, Notification, Review, Search and Analytics. Kafka provides event-driven communication. Supporting infrastructure includes Config Server, Service Discovery, databases, Redis, observability tooling, Docker and Kubernetes.

2. Service Catalog

Service	Primary Responsibility
API Gateway	Single external entry point, routing, authentication, rate limiting and resilience.
User / Identity	Registration, authentication, authorization, profiles, roles and addresses.
Product	Product catalog, categories, brands, pricing and product lifecycle.
Cart	Shopping cart management using Redis.
Order	Order creation, lifecycle, status history and business workflow.
Inventory	Stock availability, reservation, release and deduction.
Payment	Payment processing, refunds and idempotency.
Shipping	Shipment creation, tracking and delivery lifecycle.
Notification	Email, SMS and push notifications.
Review	Product ratings, comments and moderation.
Search	Product search using Elasticsearch/OpenSearch.
Analytics	Event-driven sales and business analytics.
Saga Orchestrator	Coordinates distributed order transactions and compensation.
Config Server	Centralized external configuration.
Service Discovery	Service registration and discovery, initially Eureka.

3. API Gateway Requirements

Routes: /api/v1/users/**, /products/**, /cart/**, /orders/**, /payments/** and /reviews/**. Requirements: JWT validation, user/role extraction, expired-token rejection, rate limiting, circuit-breaker integration, correlation/request IDs and CORS.

4. User / Identity Service

Entities: User, Role, Permission, Address and RefreshToken. Support registration, login, refresh token, current-user profile, address management and administrative role management. Requirements: duplicate-email detection, secure password hashing, JWT access tokens, refresh-token rotation, RBAC and Spring Security.

5. Product Service

Entities: Product, Category, Brand, ProductImage, ProductVariant and Price. Support CRUD, unique SKUs, soft deletion, pagination, sorting, category/brand/price/status filtering and keyword search. Publish ProductCreated, ProductUpdated, ProductDeleted and PriceChanged events.

6. Cart Service

Use Redis. Support add item, update quantity, remove item, view cart and clear cart. Validate product existence and active status. Apply configurable cart TTL, such as seven days. Associate carts with authenticated users.

7. Order Service

Central business service. Entities: Order, OrderItem and OrderStatusHistory. Lifecycle: CREATED → INVENTORY_RESERVATION_PENDING → INVENTORY_RESERVED → PAYMENT_PENDING → PAYMENT_SUCCESS → CONFIRMED → SHIPPED → DELIVERED. Failure states: PAYMENT_FAILED, INVENTORY_FAILED and CANCELLED. APIs: create, get, list and cancel orders. Requirements: idempotent creation, validated transitions, status history, historical product pricing and compensation.

8. Inventory Service

Fields: inventoryId, productId, warehouseId, availableQuantity, reservedQuantity, soldQuantity and version. APIs: get, reserve, release and deduct. Use optimistic locking with @Version and appropriate transaction isolation. Prevent overselling. Publish InventoryReserved, InventoryReservationFailed, InventoryReleased and InventoryDeducted.

9. Payment Service

Initially use a mock payment provider. Fields: paymentId, orderId, userId, amount, currency, status, transactionReference, idempotencyKey and timestamps. Support payment creation, lookup and refund. States: INITIATED, PROCESSING, SUCCESS, FAILED and REFUNDED. Use Idempotency-Key to prevent duplicate charges. Support timeout, retry and payment events.

10. Shipping Service

Fields: shipmentId, orderId, trackingNumber, carrier, shippingAddress, status and estimatedDeliveryDate. Support shipment creation, lookup and tracking. Lifecycle: CREATED → PICKED_UP → IN_TRANSIT → OUT_FOR_DELIVERY → DELIVERED, with RETURNED as an alternative terminal state.

11. Notification Service

Consume Kafka events including UserRegistered, OrderCreated, OrderConfirmed, PaymentSuccessful, PaymentFailed, OrderShipped and OrderDelivered. Support EMAIL, SMS and PUSH. Implement retry, Dead Letter Topics, templates and idempotent event processing.

12. Review Service

Use MongoDB. Fields: reviewId, productId, userId, orderId, rating, comment, status and createdAt. Customers may review only purchased products; normally one review per order item. Support create, list, update and delete APIs plus PENDING, APPROVED and REJECTED moderation.

13. Search Service

Use Elasticsearch/OpenSearch. Support keyword search, category/brand/price filters, sorting, pagination and autocomplete. Consume product events and maintain a search index. Demonstrate eventual consistency, CQRS and read-model synchronization.

14. Analytics Service

Consume OrderCreated, PaymentSuccessful, ProductViewed, ProductPurchased and OrderCancelled. Generate daily sales, monthly sales, top products, top categories, revenue and cancellation rate. Demonstrate event-driven analytics.

15. Saga Orchestrator

Coordinate Create Order → Reserve Inventory → Process Payment → Confirm Order → Create Shipment. If payment fails, release inventory and cancel the order. Saga state: sagaId, orderId, currentStep, status and timestamps. States: STARTED, IN_PROGRESS, COMPLETED, FAILED, COMPENSATING and COMPENSATED. Implement both choreography and orchestration for learning; use orchestration as the primary production-style implementation.

16. Kafka Requirements

Topics: user.events, product.events, order.events, inventory.events, payment.events, shipping.events and notification.events. Configure partitions, replication, consumer groups, retry topics and DLTs. Require at-least-once processing, idempotent consumers, retries and consumer monitoring. Later introduce Avro and Schema Registry.

17. Transactional Outbox

Services publishing important events should maintain business and outbox_events tables. Write both in the same database transaction, then publish asynchronously to Kafka. Track event status, retry count, published timestamp and error details.

18. Database Requirements

Follow Database-per-Service: user_db, product_db, order_db, payment_db, inventory_db and review_db. Never directly access another service's database. Communicate through REST or Kafka.

19. Observability Requirements

Propagate traceId, spanId and correlationId. Collect HTTP count, latency, error rate, Kafka consumer lag, database pool metrics, JVM memory and CPU. Use Actuator, Micrometer, Prometheus, Grafana, OpenTelemetry, Jaeger/Tempo and ELK/OpenSearch. Create dashboards for application health, Kafka, databases, infrastructure and business metrics.

20. Security Requirements

Implement authentication, authorization, RBAC, JWT, OAuth2 concepts, refresh tokens, token expiration, secure password hashing and secret management. Roles: CUSTOMER, ADMIN, SELLER, WAREHOUSE and SUPPORT. Enforce security at gateway and service level.

21. Docker Requirements

Each service must have a Dockerfile. Docker Compose should run Kafka, PostgreSQL, MongoDB, Redis, Elasticsearch/OpenSearch, Prometheus and Grafana for local development.

22. Kubernetes Requirements

Create Namespace, ConfigMap, Secret, Deployment, Service, Ingress and HPA resources. Add liveness/readiness probes, resource requests and limits, and independent horizontal scaling.

23. CI/CD Requirements

Pipeline: Git Push → Compile → Unit Tests → Integration Tests → SonarQube → Security Scan → Docker Build → Image Push → Kubernetes Deployment. Use GitHub Actions, SonarQube, Trivy, a container registry, Helm and Kubernetes.

24. Non-Functional Requirements

Availability target: 99.9%. Scalability: services scale independently. Reliability: handle service, Kafka, database and network failures, duplicate events and duplicate requests. Performance targets can start at read APIs under 500 ms and writes under 1 second, then be validated by load testing. Security: no secrets in source control, TLS in production, JWT validation, input validation, injection protection and rate limiting.

25. End-to-End Order Flow

Customer → API Gateway → Order Service → Create Order → Start Saga → Reserve Inventory → Process Payment → Confirm Order → Create Shipment → Send Notification → Order Shipped → Order Delivered. Failure path: Payment Failed → Release Inventory → Cancel Order. This demonstrates REST, Kafka, Saga, Outbox, idempotency, transactions, eventual consistency, circuit breakers, retries, distributed tracing, security and observability.

26. Recommended Development Milestones

1) User + Product + PostgreSQL + REST + JPA. 2) API Gateway + JWT + Spring Security. 3) Order + Inventory + Payment. 4) OpenFeign + service communication + Eureka. 5) Kafka + events + consumers + consumer groups. 6) Saga + workflow + compensation. 7) Outbox + idempotency + retry + DLT. 8) Redis + Cart + caching. 9) MongoDB + Review + Elasticsearch/OpenSearch. 10) Resilience4j + Circuit Breaker + Bulkhead + Rate Limiting. 11) Docker + Compose + Kubernetes + Helm. 12) Prometheus + Grafana + OpenTelemetry + CI/CD + AWS.

27. Final Architecture Learning Path

REST → Microservices → Gateway → Discovery → Kafka → Saga → Outbox → Idempotency → Resilience → Security → Observability → Docker → Kubernetes → CI/CD → AWS

28. Core Business Workflow

Customer → Cart → Order → Inventory Reservation → Payment → Order Confirmation → Shipping → Notification.

Architectural backbone: API Gateway + JWT + REST + Kafka + Saga + Outbox + Idempotency + Resilience4j + PostgreSQL + Redis + MongoDB + Elasticsearch/OpenSearch + Docker + Kubernetes + Observability + CI/CD.